

Abstract

Usual methods of sampling rare events for the purpose of quantifying their return times are either computationally expensive, time consuming or both. Here this is addressed by coupling a Global Climate Model with a neural network trained on long runs of the same model, to guide the sampling in an efficient way.

Background

- Global Climate Model (GCM): a physics-based model that works by numerically solving differential equations of climate systems.

- Estimating probabilities of rare events (like heatwaves, floods, cyclones) is difficult because we have limited samples for them. Usually a large number of simulations are required.

- A naive approach could be Direct Numerical Simulation (DNS): run the GCM for extended periods starting from a number of perturbed initial conditions. But this is very computationally expensive and time consuming.

- A better approach for Rare Event Sampling (RES) could be: stop the simulation at certain timesteps. Figure out which trajectories are ‘promising’ and clone them. Similarly, terminate the trajectories which are unlikely or less promising. This will naturally guide the simulation towards event of interest.

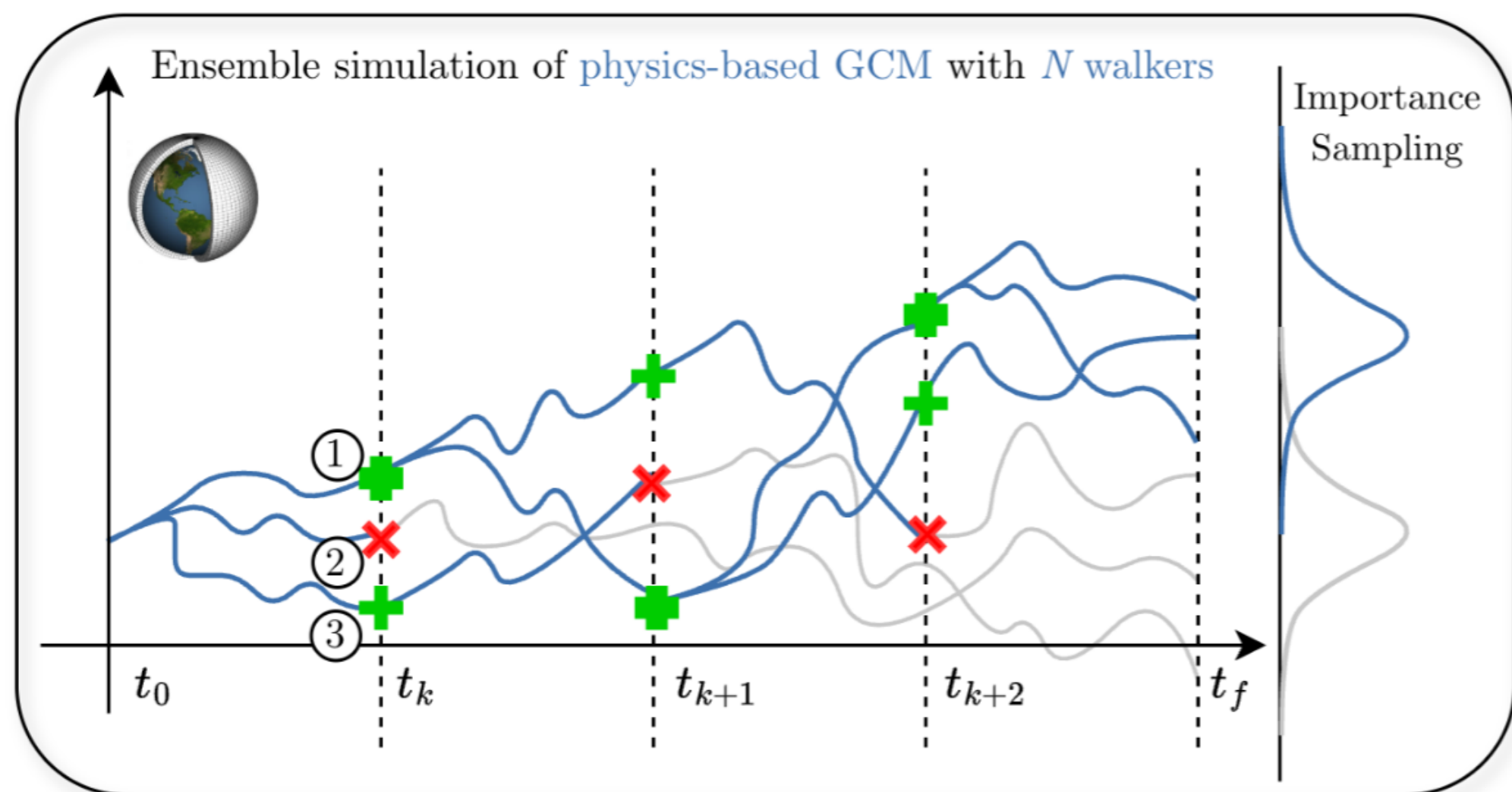


Figure 1: Schematic illustration of Rare Event Sampling (RES)

- RES efficiency depends on a *score function* to rank trajectories. Constructing this can be highly difficult in high-dimensional chaotic systems; requires domain knowledge.

- Although the above process can be done only with a GCM, it would take a lot of time. Machine learning based weather prediction models generally show decent skill upto 15-20 days.

- So here we couple both of them: the GCM evolves the trajectories, and the AI emulator – which is trained on long-runs of the same GCM – makes short-term forecasts which are used to score those trajectories (or walkers).

About the model

Training objective: given X_t, X_{t-1} , predict X_{t+1} .

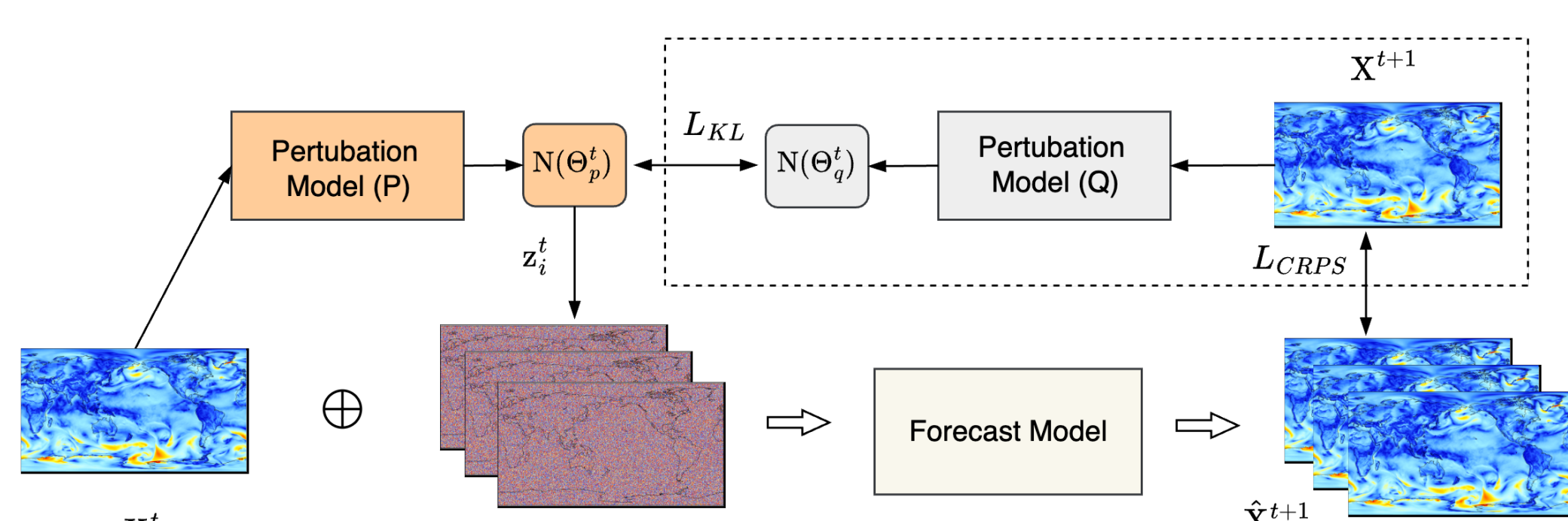
Data

- 260 (training) + 60 (validation) years of daily data from runs of the GCM

- 14 variables: surface air temperature, surface air pressure; and air temperature, specific humidity, eastward wind, northward wind at 3 pressure levels (860, 500, 260 hPa).

- 64 latitudes x 128 longitudes (2.8125 degrees); channel-wise normalized (z-scored)

Architecture



Training

- Stage 1: only the forecasting model is pre-trained using latitude-weighted L_1 loss

$$L_1 = \frac{1}{CHW} \sum_{c=1}^C \sum_{i=1}^H \sum_{j=1}^W a_i \left| \hat{X}_{c,i,j}^{t+1} - X_{c,i,j}^{t+1} \right|$$

where

$$a_i = \frac{H \cos \Phi_i}{\sum_{i=1}^H \cos \Phi_i}$$

is the latitude-dependent weight at latitude Φ_i .

- Stage 2: the perturbation model and forecasting model are jointly trained to minimize a combination of CRPS and KL divergence:

$$L = L_{\text{CRPS}}(\hat{X}_i^{t+1}, X_i^{t+1}) + \lambda L_{\text{KL}}[\mathcal{N}(\Theta_p^t), \mathcal{N}(\Theta_q^t)]$$

where $\lambda = 10^{-4}$

Here

$$\text{CRPS} = \frac{1}{N} \sum_{i=1}^N \left| \hat{Y}_i - Y \right| - \frac{1}{2N^2} \sum_{i=1}^N \sum_{j=1}^N \left| \hat{Y}_i - \hat{Y}_j \right|$$

Evaluation

- Anomaly Correlation Coefficient (ACC):

$$\text{ACC} = \frac{\sum_{i=1}^{N_{\text{lat}}} \sum_{j=1}^{N_{\text{lon}}} f'_{i,j} a'_{i,j}}{\sqrt{\left(\sum_{i=1}^{N_{\text{lat}}} \sum_{j=1}^{N_{\text{lon}}} (f'_{i,j})^2 \right) \left(\sum_{i=1}^{N_{\text{lat}}} \sum_{j=1}^{N_{\text{lon}}} (a'_{i,j})^2 \right)}}$$

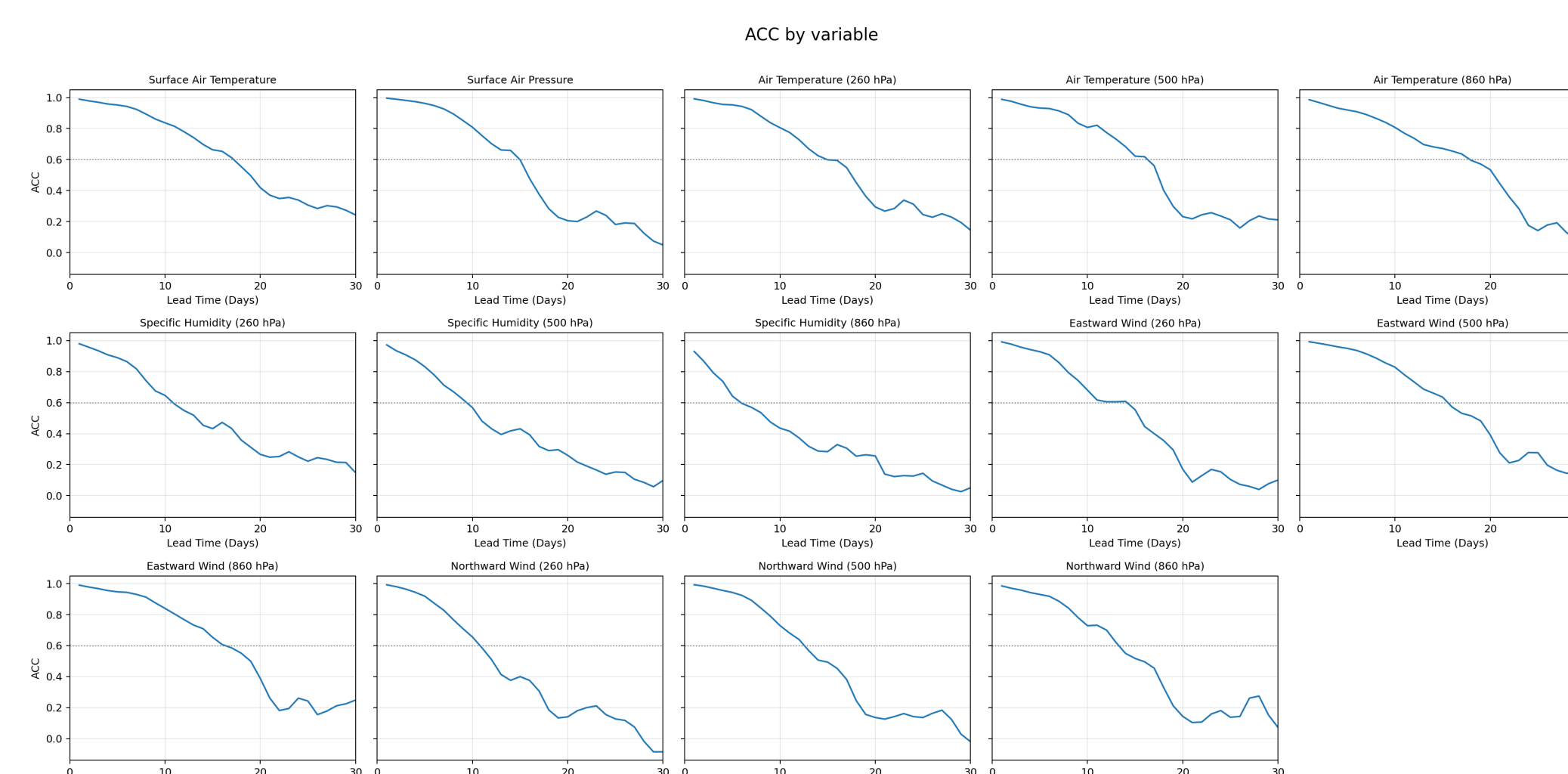


Figure 2: ACC vs lead time for all 14 variables.

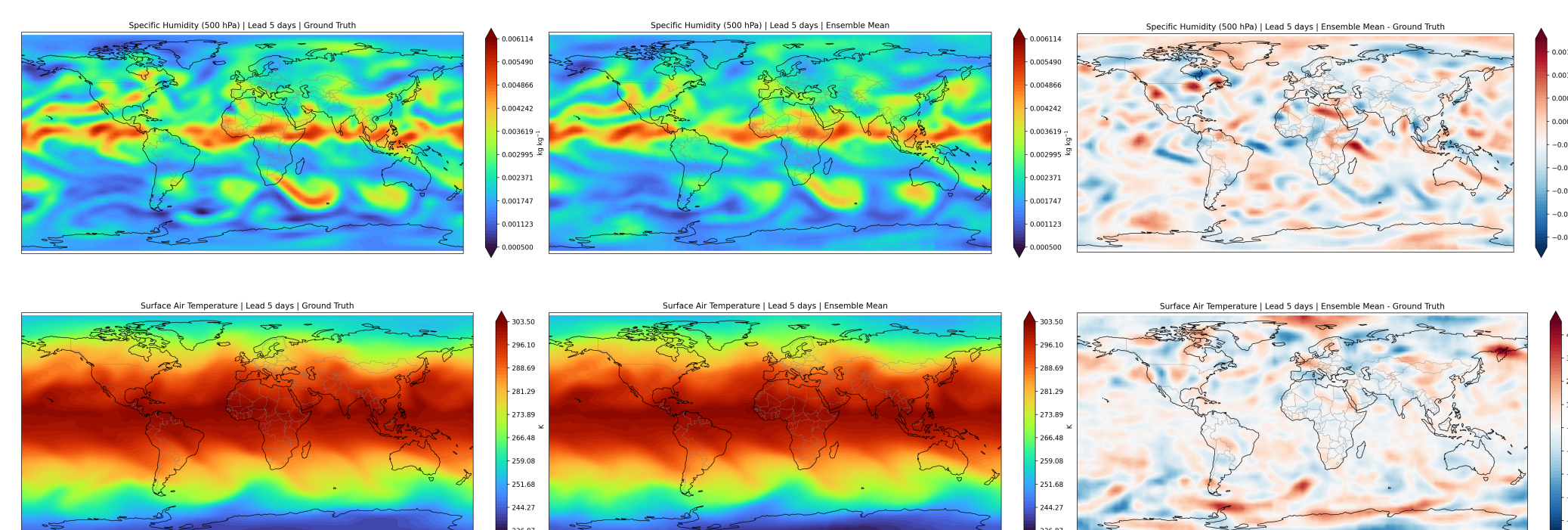


Figure 3: Field comparison for specific humidity (top; 500 hPa) and surface temperature (bottom)

Resampling algorithm

C_k (splitting scheme): [0.2, 0.4, 0.7, 1.8, 2.2, 2.6]

Require: N walkers, K steps, AI emulator $\mathcal{E}$, Physics

$\mathcal{M}$, parameter C_0

```

1: Initialize ensemble  $\{X_0^i\}_{i=1}^N$ 
2: for  $k = 0$  to  $K - 1$  do
3:   % AI Emulator Scoring
4:   for  $i = 1$  to  $N$  do
5:     Generate  $M$  forecasts from  $X_k^i$  to  $t_f$  using  $\mathcal{E}$ 
6:     Compute score  $\theta_k^i$  as ensemble mean over
       target observable
7:     Compute weight  $w_k^i = \exp(C_0 \theta_k^i)$  if  $k = 0$  else
        $\bar{w}_{k-1} \exp(C_k \theta_k^i - C_{k-1} \hat{\theta}_{k-1}^i)$ 
8:   end for
9:   for  $i = 1$  to  $N$  do
10:    Normalized weight  $\tilde{w}_k^i = N w_k^i / \sum w_k^j$ 
11:     $N_k^i = \lfloor \tilde{w}_k^i \rfloor$ ,  $\delta_k^i = \tilde{w}_k^i - N_k^i$ 
12:  end for

```

```

13: % Pivotal resampling
14: Let  $U$  be active indices where  $0 < \delta_k^i < 1$ 
15: while  $|U| > 1$  do
16:   Select  $a, b \in U$  and compute  $s = \delta_k^a + \delta_k^b$ 
17:   if  $s \leq 1$  then
18:     w.p.  $\delta_k^a/s$ :  $\delta_k^a \leftarrow s$ ,  $\delta_k^b \leftarrow 0$ 
19:     w.p.  $\delta_k^b/s$ :  $\delta_k^a \leftarrow 0$ ,  $\delta_k^b \leftarrow s$ 
20:   else if  $s > 1$  then
21:     w.p.  $\frac{1-\delta_k^b}{2-s}$ :  $\delta_k^a \leftarrow 1$ ,  $\delta_k^b \leftarrow s - 1$ 
22:     w.p.  $\frac{1-\delta_k^a}{2-s}$ :  $\delta_k^a \leftarrow s - 1$ ,  $\delta_k^b \leftarrow 1$ 
23:   end if
24:   if  $\delta_k^a \in \{0, 1\}$  then  $U \leftarrow U \setminus \{a\}$ ,
      $N_k^a \leftarrow N_k^a + \delta_k^a$ 
25:   end if
26:   if  $\delta_k^b \in \{0, 1\}$  then  $U \leftarrow U \setminus \{b\}$ ,
      $N_k^b \leftarrow N_k^b + \delta_k^b$ 
27:   end if
28: end while
29: % Now back to the GCM
30: Form  $\{\hat{X}_k^i\}_{i=1}^N$  by cloning each  $X_k^i$  exactly  $N_k^i$  times
31: for  $j = 1$  to  $N$  do
32:   Perturb  $\hat{X}_k^j$  and integrate:  $X_{k+1}^j = \mathcal{M}(\hat{X}_k^j)$ 
33: end for
34: end for
Ensure: Final ensemble  $\{X_K^i\}_{i=1}^N$ 

```

Results

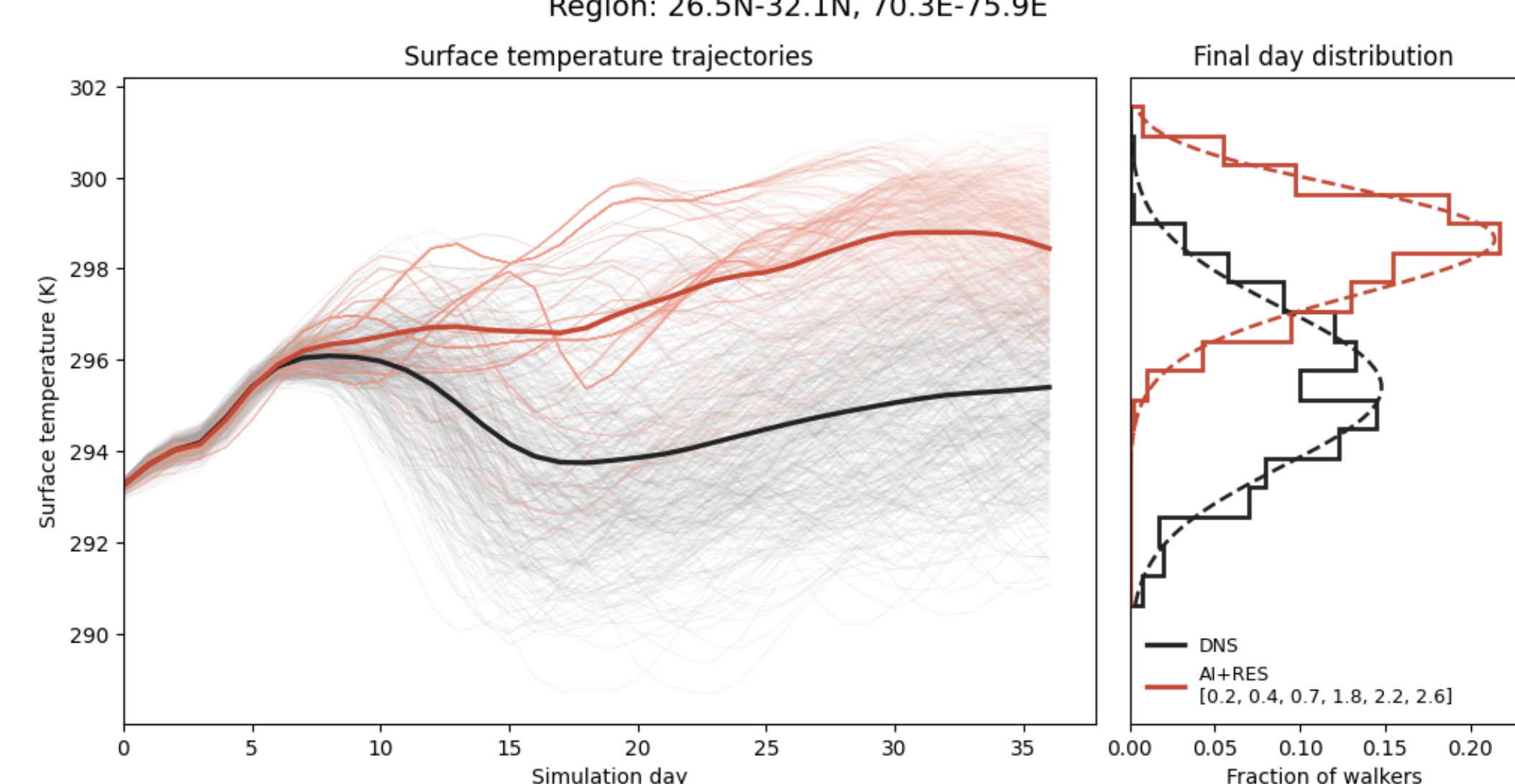


Figure 4: Evolution of individual walkers for both setups

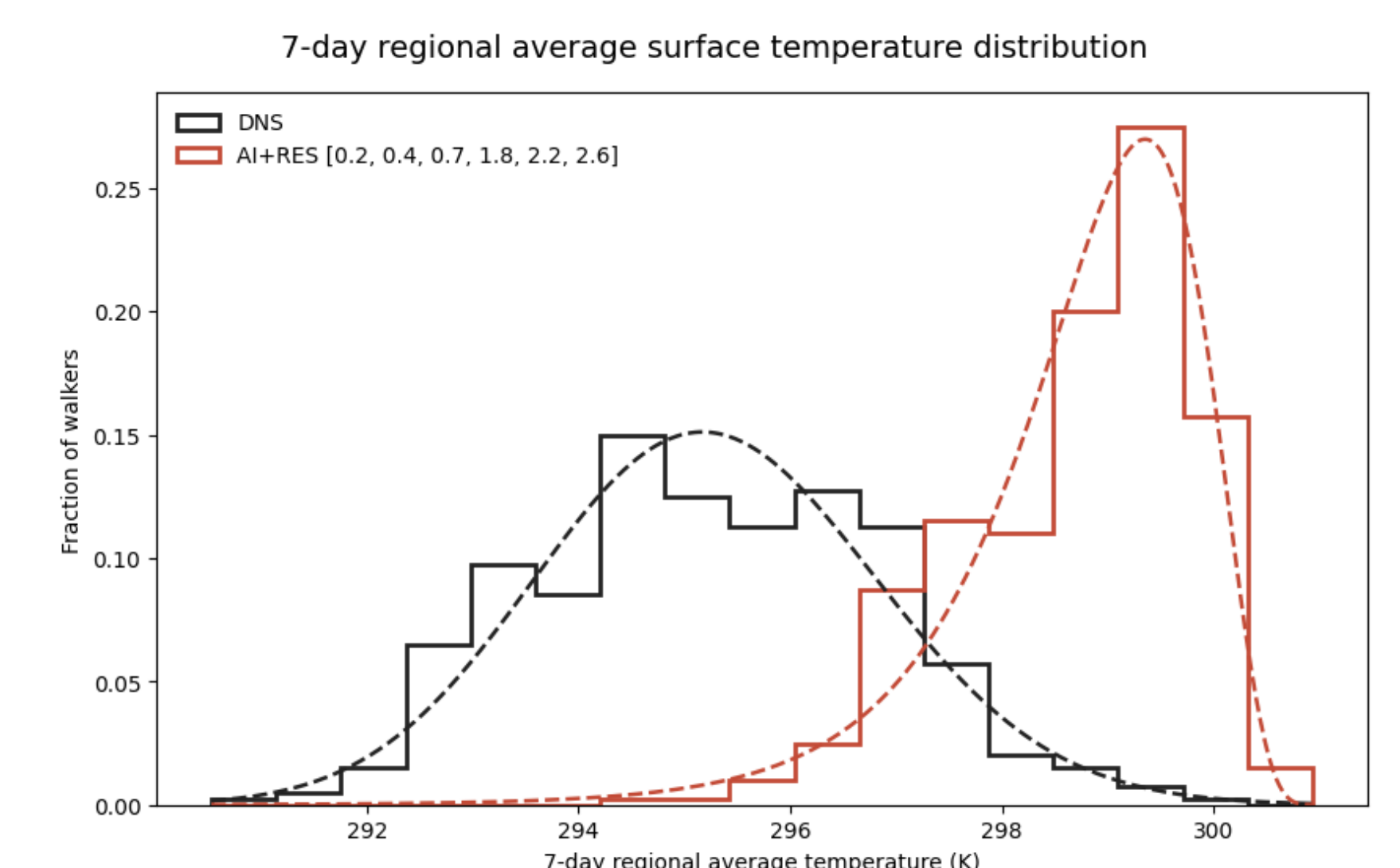


Figure 5: Distribution of the observable

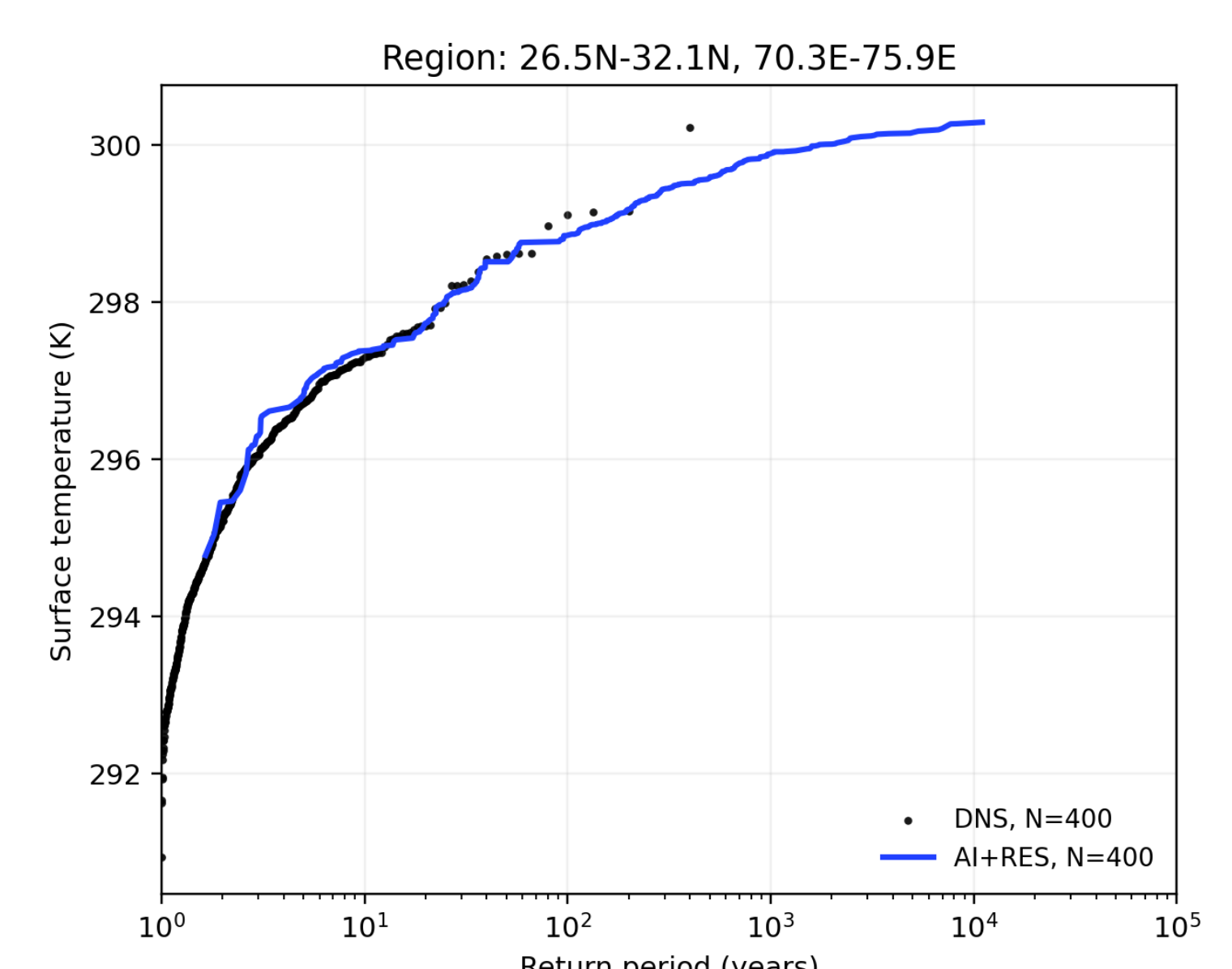


Figure 6: Return-period curves

References

1. Lancelin, A., et al. (2025). *AI-boosted rare event sampling*. arXiv:2510.27066.
2. Zhong, X., et al. (2024). *FuXi-ENS*. arXiv:2405.05925.